Apache Kafka

What is Kafka?

Kafka transfers data fast between systems — acting as both a messaging system and streaming platform. It decouples systems so they communicate via Kafka rather than directly (A → Kafka ← B).

Who Uses Kafka?

- LinkedIn — handle activity data & operational metrics
- Square — move data across data centers

Kafka Projects

- Better than traditional systems
- Real-time stream
- Integrate with systems
- Feed data lake
- Kafka Streams (processing in Kafka itself as data flows)

Kafka APIs

- Producer
- Consumer
- Streams
- Connector
- Admin (manage clusters & topics)

Use Cases

- Stream processing & analytics
- Data pipeline for big systems
- CQRS (replay events)
- Real-time stream integration
- Feed data lake
- Parallel reading to speed up data ingestion in Hadoop clusters
- Kafka Streams integration (processing in Kafka itself as data flows)

Advantages & Disadvantages

Advantages	Disadvantages
• Fast/Performant (+100k msg/sec) • Low latency (fast = small delay) • Scalable (add servers) • Fault-tolerant (replication) → reliability + availability • Message partitioning & ordering → scalability + consistency • Durable & Persistent (stored on disk) • Stream & Batch processing • Producer Control/Tunable Consistency (speed ↔ reliability tradeoff) • Consumer Friendly (can control its data) • High Concurrency (producer || consumer) • Flexible (Multiple msg patterns) • Easy Setup	• No monitoring • Immutable messages (after write) • Topic selection not random (topic name need to be known) • Complex 4 beginners • Slower (disk slow, cache faster) • Weaker Performance (on bad configuration only) • Messaging Patterns not all included

Why is Kafka Fast?

- 0-copy (kernel-level data transfer)
- Batching (grouped messages processing)
- Horizontal Scaling (many partitions per topic)
- Ordered Disk Writes (no random access)

Messaging Patterns

1. P2P (point-to-point): 1 producer & 1 consumer only
2. Publish-Subscribe: 1 producer & many consumers

Kafka Components

Component	Role	Notes
Records	Immutable data unit	Fields: key, value, timestamp
Topics	Named record streams	—
Producers	Data writers	Choose partition; round-robin or custom
Consumers	Data readers	Pull-based; low-level or consumer groups
Brokers	Servers / Clusters	Handle reads/writes; leader + followers
Connectors	System integration	Source (external → Kafka) / Sink (Kafka → external)
Streamers	Stream processing	Filter, transform, aggregate
ZooKeeper	Cluster manager	—

Producers

- Choose partition (round-robin or custom logic)
- Send as fast as broker allows
- No waiting for broker's confirmation

Consumers

- Read messages from a specific or any offset
- Low-level consumer = manual choosing
- High-level consumer = consumer groups
- In a group, 1 partition per consumer
- Kafka doesn't push data to consumers
- Pull data when ready & catch up later (queues)

Streams & Kafka Streams

Streams

- Ordered data split into partitions (same as Kafka topic partitions)
- Record = Kafka message
- Key decides partition (keeps order + grouping)
- Kafka Streams splits processing into tasks (1 task per partition)

Kafka Streams

Java library for processing Kafka data — embedded in Java apps, built on Kafka.

Features

- Stateful ops (aggregations, windowing, local state)
- Exactly-once processing (no duplicates)
- Event-time windowing (based on timestamps)
- Flexible APIs (DSL or low-level processor API)

Threading Model

- Threads run tasks in a Kafka Streams app
- Each thread executes one or more tasks

Local State Store

- Keeps states that a task needs while processing
- Supports stateful operations (aggregate, join, window) embedded inside each task
- Features: fault-tolerance, recovery, queryable, efficiency

Streaming Pipeline

- Producers → Kafka (collects & streams)
- Kafka → Spark (processes)
- Spark → Storage (saves processed data)
- Storage → Applications (use data)

Stream Processors	Source Processor	Sink Processor
Role	Input stream	Output stream
Function	Reads from kafka topics & sends data	Receives data & writes to kafka topics

Key Notes

Offset = position in partition = ID/Serial	Performance is constant
Records kept for set time	Offset controlled by consumer decides update time
Partition size must fit on a server	Messages with same keys → same partitions
Leader = handles reads/writes	Follower = replicates data (no requests)
A server is a leader & follower for different partitions	Replication factor = 3 → 1 leader + 2 followers
RF ≤ servers; ↑ replicas = ↑ fault tolerance	↓ replicas = ↓ fault tolerance, ↓ overhead
Stream logic = topology (graph of processors + data flow)	Stream Processor: take record, apply operation, output record